C# / .NET STUDY NOTES

Conditional Statements and Loops in C#

Roadmap Topic 4

Learning focus
Program control flow, conditions, branching, loops, guard clauses, iteration, and practical Fundraising Platform processing.

Examples
Gift classification, validation, retries, collection processing, authorization conditions, batching, and loop behavior.

Supplied study notes → structured sections → examples and flows → revision and interview preparation

Contents

Click a section name to jump directly to that part of the notes.

01 Topic Coverage
02 What Is Control Flow?
03 Definition
04 What Is a Condition?
05 if Statement
06 Definition
07 Fundraising Example
08 if Requires a Boolean Expression
09 Curly Braces
10 if-else Statement
11 Definition
12 Fundraising Example
13 else if
14 Order of Conditions Matters
15 Nested if Statements
16 Nested Conditions Can Become Hard to Read
17 Guard Clauses
18 Definition
19 Guard Clause in a Method
20 Combining Conditions
21 Do Not Over-Combine Conditions
22 switch Statement
23 Definition
24 case
25 default
26 break in switch
27 Multiple Cases with Same Behavior
28 switch with Strings
29 switch vs if-else
30 Switch Expression
31 Switch Expression Flow
32 When Switch Expressions Are Good
33 Avoid Huge Switch Expressions
34 Pattern Matching with switch
35 Pattern Order Matters
36 for Loop
37 Definition
38 Parts of a for Loop
39 for Loop Flow
40 Fundraising Example
41 Array Indexes Start at Zero
42 Off-by-One Error
43 Counting Down with for
44 Different Increment Sizes
45 Multiple Variables in for
46 foreach Loop
47 Definition
48 foreach Syntax
49 Fundraising Example
50 foreach vs for
51 foreach and Index
52 Modifying Collection During foreach

53 Safe Alternatives
54 while Loop
55 Definition
56 while Flow
57 Example Where while Executes Zero Times
58 When to Use while
59 Fundraising Retry Example
60 do-while Loop
61 Definition
62 do-while Flow
63 while vs do-while
64 while
65 do-while
66 Example
67 When to Use do-while
68 Nested Loops
69 Nested Loop Cost
70 break Statement
71 Definition
72 When break Is Useful
73 break Only Exits Nearest Loop
74 continue Statement
75 Definition
76 break vs continue
77 return Statement
78 Definition
79 return with Value
80 break vs continue vs return
81 Infinite Loop
82 Infinite Loop Can Be Intentional
83 Accidental Infinite Loop
84 Loop Variable Scope
85 foreach Variable Scope
86 Variable Shadowing
87 Iterating Arrays
88 Iterating List
89 Iterating Dictionary<TKey,TValue>
90 Choosing for vs foreach vs while vs do-while
91 Use for when
92 Use foreach when
93 Use while when
94 Use do-while when
95 Practical Fundraising Example — Process Valid Gifts
96 Practical Example — Find First Major Gift
97 Practical Example — Calculate Total Giving
98 Practical Example — Count Major Gifts
99 Practical Example — Gift Classification
100 Practical Example — Batch Processing
101 Practical Example — Retry with break
102 Avoid Blocking Retry Loops in Real Web APIs
103 Conditions and Null Handling
104 Pattern Matching in if
105 Multiple Conditions with Pattern Matching
106 Common Mistake — Wrong Condition Order
107 Common Mistake — Missing Braces
108 Common Mistake — Assignment vs Equality

109 Common Mistake — Infinite while Loop
110 Common Mistake — Off-by-One Error
111 Common Mistake — Modifying Collection in foreach
112 Common Mistake — Too Much Nesting
113 Common Mistake — continue Everywhere
114 Common Mistake — break When return Is Needed
115 Performance Considerations
116 Algorithmic Cost Matters
117 Avoid Database Calls Inside Large Loops
118 Async Loop Preview
119 Security Considerations
120 Conditions Are Not a Substitute for Authorization
121 Practical Coding Exercise — Even or Odd
122 Practical Coding Exercise — Largest of Two Numbers
123 Practical Coding Exercise — Largest of Three Numbers
124 Practical Coding Exercise — Sum Array
125 Practical Coding Exercise — Find Maximum
126 Practical Coding Exercise — Count Positive Values
127 Practical Coding Exercise — Reverse Output
128 Practical Coding Exercise — Palindrome Preview
129 Interview Questions
130 What is control flow?
131 What is an if statement?
132 What is the difference between if and if-else?
133 What is an else-if chain?
134 Why does condition order matter in else-if?
135 What is a guard clause?
136 What is a switch statement?
137 What is a switch expression?
138 When should you use switch instead of if?
139 What is a for loop?
140 What is foreach?
141 Difference between for and foreach?
142 What is a while loop?
143 What is a do-while loop?
144 What is the difference between while and do-while?
145 What does break do?
146 What does continue do?
147 What does return do?
148 Difference between break and return?
149 What is an infinite loop?
150 What is an off-by-one error?
151 Can you modify a List while iterating with foreach?
152 What is a nested loop?
153 Which loop is most common in business C# code?
154 Best Practices
155 Quick Revision
156 Complete Fundraising Example
157 Complete Topic Flow

Topic Coverage

Conditional statements and loops control the **flow of a C# program**.

Until now, we learned:

```
Variables
    →
Data Types
    →
Operators
    →
Expressions
```

Now we use those expressions to decide:

```
Should this code execute?
```

and:

```
How many times should this code repeat?
```

This topic covers:

- program control flow;
- Boolean conditions;
- `if`;
- `else`;
- `else if`;
- nested conditions;
- guard clauses;
- `switch`;
- traditional `switch` statements;
- switch expressions;
- pattern matching inside conditions;
- `for`;
- `foreach`;
- `while`;
- `do while`;
- nested loops;
- `break`;
- `continue`;
- `return`;
- infinite loops;
- loop variable scope;
- iteration over arrays and collections;
- modifying collections during iteration;

- choosing the correct loop;
- performance considerations;
- common mistakes;
- Fundraising Platform examples;
- interview questions;
- practical coding exercises.

The main flow is:

```
Program starts
    →
Evaluate condition
    →
Choose path
    →
Possibly repeat work
    →
Continue program
```

What Is Control Flow?

Definition

Control flow means the order in which statements in a program execute.

Without conditions or loops, code normally runs from top to bottom.

Example:

```
Console.WriteLine("Start");

decimal giftAmount = 1000m;

Console.WriteLine(giftAmount);

Console.WriteLine("End");
```

Execution:

```
Start
    →
Create giftAmount
    →
Print giftAmount
    →
End
```

But real applications need decisions.

For example:

```
Gift amount > 0?
    →
Yes → Process gift

No → Reject gift
```

That requires conditional control flow.

What Is a Condition?

A condition is an expression whose result is:

```
true
```

or:

```
false
```

Example:

```
decimal giftAmount = 1000m;

bool isValid =
    giftAmount > 0m;
```

The expression:

```
giftAmount > 0m
```

is the condition.

It produces a Boolean result.

Conditions are used by:

```
if
while
do while
for
conditional operator
```

and many other parts of C#.

---

if Statement

Definition

An `if` statement executes a block of code only when a Boolean condition is `true`.

Syntax:

```
if (condition)
{
    // code
}
```

Example:

```
decimal giftAmount = 1000m;

if (giftAmount > 0m)
{
    Console.WriteLine("Valid gift amount");
}
```

Flow:

```
giftAmount > 0?
    →
true
    →
execute block
```

If the condition is false:

```
skip block
```

Fundraising Example

```
decimal giftAmount = 5000m;

if (giftAmount >= 5000m)
{
    Console.WriteLine("Major gift");
}
```

Since:

```
5000 >= 5000
```

is true, the message is printed.

if Requires a Boolean Expression

C# does not allow:

```
// if (1)
// {
// }
```

The condition must produce:

```
bool
```

Correct:

```
if (giftCount > 0)
{
}
```

This is because C# does not automatically treat:

```
0 = false
1 = true
```

for normal conditions.

Curly Braces

You can technically write:

```
if (giftAmount > 0m)
    Console.WriteLine("Valid");
```

But for professional code, prefer:

```
if (giftAmount > 0m)
{
    Console.WriteLine("Valid");
}
```

Why?

Because future changes are safer.

Consider:

```
if (giftAmount > 0m)
    Console.WriteLine("Valid");
    ProcessGift();
```

Only the first statement belongs to the `if`.

`ProcessGift()` runs every time.

Using braces avoids this mistake.

---

if-else Statement

Definition

`if-else` chooses between two paths.

Syntax:

```
if (condition)
{
    // true path
}
else
{
    // false path
}
```

Example:

```
decimal giftAmount = 1000m;

if (giftAmount > 0m)
{
    Console.WriteLine("Gift accepted");
}
else
{
    Console.WriteLine("Gift rejected");
}
```

Flow:

```
GiftAmount > 0?
    →
Yes
    →
Accept Gift

No
    →
Reject Gift
```

Only one branch executes.

Fundraising Example

```
bool isAnonymous = true;

if (isAnonymous)
{
    Console.WriteLine("Anonymous Donor");
}
else
{
    Console.WriteLine("Named Donor");
}
```

---

else if

Use `else if` when more than two conditions are possible.

Example:

```
decimal giftAmount = 7500m;

if (giftAmount >= 10_000m)
{
    Console.WriteLine("Principal Gift");
}
else if (giftAmount >= 5_000m)
{
    Console.WriteLine("Major Gift");
}
else if (giftAmount >= 1_000m)
{
    Console.WriteLine("Leadership Gift");
}
else
{
    Console.WriteLine("Standard Gift");
}
```

Flow:

```
Amount >= 10000?
    →
No
    →
Amount >= 5000?
    →
Yes
    →
Major Gift
```

Once a matching branch is found:

```
later branches are skipped
```

Order of Conditions Matters

This is very important.

Correct:

```
if (giftAmount >= 10_000m)
{
    Console.WriteLine("Principal Gift");
}
else if (giftAmount >= 5_000m)
{
    Console.WriteLine("Major Gift");
}
```

Wrong order:

```
if (giftAmount >= 5_000m)
{
    Console.WriteLine("Major Gift");
}
else if (giftAmount >= 10_000m)
{
    Console.WriteLine("Principal Gift");
}
```

If:

```
giftAmount = 15000
```

the first condition:

```
>= 5000
```

is already true.

So the more specific:

```
>= 10000
```

branch is never reached.

Rule:

```
More specific conditions
    →
usually check first

Broader conditions
    →
later
```

Nested if Statements

An `if` can appear inside another `if`.

Example:

```
if (giftAmount > 0m)
{
    if (isCampaignActive)
    {
        Console.WriteLine("Process gift");
    }
}
```

Flow:

```
Gift amount valid?
    →
Yes
    →
Campaign active?
    →
Yes
    →
Process Gift
```

Nested Conditions Can Become Hard to Read

Example:

```
if (giftAmount > 0m)
{
    if (constituentId > 0)
    {
        if (isCampaignActive)
        {
            if (paymentSucceeded)
            {
                ProcessGift();
            }
        }
    }
}
```

This works but creates:

```
deep nesting
```

which makes code harder to understand.

A better approach may use:

```
guard clauses
```

Guard Clauses

Definition

A **guard clause exits early when a required condition is not satisfied**.

Instead of deeply nesting valid cases, reject invalid cases first.

Example:

```
if (giftAmount <= 0m)
{
    return;
}

if (constituentId <= 0)
{
    return;
}

if (!isCampaignActive)
{
    return;
}

if (!paymentSucceeded)
{
    return;
}

ProcessGift();
```

Flow:

```
Invalid amount?
    →
Yes → Exit

No
    →
Invalid constituent?
    →
Yes → Exit

No
    →
Inactive campaign?
    →
Yes → Exit

No
    →
Payment failed?
    →
Yes → Exit

No
    →
Process Gift
```

This is often easier to read than deeply nested `if` statements.

Guard Clause in a Method

```
static void ProcessGift(
    decimal giftAmount,
    bool paymentSucceeded)
{
    if (giftAmount <= 0m)
    {
        return;
    }

    if (!paymentSucceeded)
    {
        return;
    }

    Console.WriteLine("Gift processed");
}
```

Later, we will often use:

```
exceptions
validation results
ProblemDetails
```

instead of simply returning.

But the control-flow idea remains useful.

Combining Conditions

Instead of nesting:

```
if (giftAmount > 0m)
{
    if (paymentSucceeded)
    {
        ProcessGift();
    }
}
```

you can write:

```
if (giftAmount > 0m
    &&
    paymentSucceeded)
{
    ProcessGift();
}
```

This is useful when the conditions form one clear business rule.

Do Not Over-Combine Conditions

This becomes difficult:

```
if ((a && b || c)
    && (!d || e)
    && f
    && (g || h && i))
{
}
```

Even if technically correct, it is difficult to maintain.

Instead:

```
bool validGift =
    amountValid
    &&
    paymentSucceeded;

bool authorized =
    isAdmin
    ||
    isGiftOwner;

if (validGift && authorized)
{
}
```

Readable intermediate Boolean variables are often valuable.

switch Statement

Definition

A `switch` statement selects one branch from several possible cases.

It is useful when one value is compared against several distinct possibilities.

Example:

```
int giftType = 2;

switch (giftType)
{
    case 1:
        Console.WriteLine("Cash");
        break;

    case 2:
        Console.WriteLine("Check");
        break;

    case 3:
        Console.WriteLine("Electronic");
        break;

    default:
        Console.WriteLine("Other");
        break;
}
```

Flow:

```
giftType
    →
1? Cash
2? Check
3? Electronic
otherwise Other
```

---

case

Each `case` represents one possible match.

```
case 1:
```

means:

```
When switch value matches 1
```

execute that branch.

---

default

`default` handles values that do not match previous cases.

```
default:
    Console.WriteLine("Unknown type");
    break;
```

Think of `default` as similar to:

```
else
```

in an `if` structure.

break in switch

Traditional `switch` branches commonly end with:

```
break;
```

This exits the `switch`.

Example:

```
case 1:
    Console.WriteLine("Cash");
    break;
```

Without appropriate branch termination, the compiler prevents accidental C-style fall-through between normal non-empty cases.

Multiple Cases with Same Behavior

Example:

```
switch (giftType)
{
    case 1:
    case 2:
        Console.WriteLine("Offline Payment");
        break;

    case 3:
        Console.WriteLine("Electronic Payment");
        break;
}
```

Both:

```
1
2
```

use the same branch.

switch with Strings

```
string currencyCode = "USD";

switch (currencyCode)
{
    case "USD":
        Console.WriteLine("US Dollar");
        break;

    case "EUR":
        Console.WriteLine("Euro");
        break;

    case "GBP":
        Console.WriteLine("British Pound");
        break;

    default:
        Console.WriteLine("Unknown Currency");
        break;
}
```

switch vs if-else

Use `if` when conditions are based on:

```
ranges
multiple variables
complex Boolean logic
```

Example:

```
if (giftAmount >= 5000m
    && isCampaignActive)
{
}
```

Use `switch` when one value is being matched against several alternatives.

Example:

```
GiftType
    →
Cash
Check
Electronic
Other
```

Switch Expression

Modern C# supports **switch expressions**.

Instead of:

```
string description;

switch (giftType)
{
    case 1:
        description = "Cash";
        break;

    case 2:
        description = "Check";
        break;

    case 3:
        description = "Electronic";
        break;

    default:
        description = "Other";
        break;
}
```

you can write:

```
string description =
    giftType switch
    {
        1 => "Cash",
        2 => "Check",
        3 => "Electronic",
        _ => "Other"
    };
```

Switch Expression Flow

```
giftType
    →
match 1?
    →
"Cash"

match 2?
    →
"Check"

match 3?
    →
"Electronic"

anything else
    →
"Other"
```

The underscore:

```
_
```

acts as a discard/default match.

When Switch Expressions Are Good

Use them when:

```
one input
    →
maps cleanly
    →
to one output value
```

Example:

```
string statusDescription =
    statusCode switch
    {
        1 => "Pending",
        2 => "Completed",
        3 => "Cancelled",
        _ => "Unknown"
    };
```

This is concise and readable.

Avoid Huge Switch Expressions

Do not put large business workflows inside a switch expression.

Bad:

```
20 cases
each with complex method calls
nested expressions
multiple side effects
```

A switch expression works best for clear mapping logic.

Pattern Matching with switch

Modern C# switch expressions can match more than exact constants.

Example:

```
string category =
    giftAmount switch
    {
        >= 10_000m => "Principal",
        >= 5_000m => "Major",
        >= 1_000m => "Leadership",
        > 0m => "Standard",
        _ => "Invalid"
    };
```

This uses **relational patterns**.

Flow:

```
Amount >= 10000?
    →
Principal

otherwise >= 5000?
    →
Major

otherwise >= 1000?
    →
Leadership

otherwise > 0?
    →
Standard

otherwise
    →
Invalid
```

Pattern matching will be covered deeply in a later roadmap topic.

Pattern Order Matters

Correct:

```
giftAmount switch
{
    >= 10_000m => "Principal",
    >= 5_000m => "Major",
    >= 1_000m => "Leadership",
    _ => "Standard"
};
```

If you put the broader pattern first:

```
>= 1_000m
```

then larger values could already match that branch.

Always think:

```
specific
    →
general
```

for Loop

Definition

A `for` loop repeats code while a condition remains true and is especially useful when the number of iterations is known or controlled by a counter.

Syntax:

```
for (initialization; condition; iteration)
{
    // repeated code
}
```

Example:

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Output:

```
0
1
2
3
4
```

Parts of a for Loop

```
for (int i = 0; i < 5; i++)
```

contains three major parts.

Initialization:

```
int i = 0
```

Condition:

```
i < 5
```

Iteration:

```
i++
```

for Loop Flow

```
Initialize i = 0
    →
Check i < 5
    →
true
    →
Execute body
    →
i++
    →
Check condition again
    →
repeat
```

When condition becomes false:

```
loop ends
```

Fundraising Example

Suppose:

```
decimal[] gifts =
{
    100m,
    250m,
    500m
};
```

Using `for`:

```
for (int i = 0; i < gifts.Length; i++)
{
    Console.WriteLine(gifts[i]);
}
```

Flow:

```
i = 0 → gifts[0]
i = 1 → gifts[1]
i = 2 → gifts[2]
i = 3 → condition false → stop
```

Array Indexes Start at Zero

For:

```
decimal[] gifts =
{
    100m,
    250m,
    500m
};
```

indexes are:

```
0 → 100
1 → 250
2 → 500
```

Therefore:

```
i < gifts.Length
```

is usually correct.

Do not write:

```
i <= gifts.Length
```

because the final iteration would attempt:

```
gifts[gifts.Length]
```

which is outside the array.

Off-by-One Error

Common mistake:

```
for (int i = 0; i <= gifts.Length; i++)
{
    Console.WriteLine(gifts[i]);
}
```

Suppose:

```
Length = 3
```

valid indexes:

```
0
1
2
```

But this loop eventually reaches:

```
i = 3
```

and tries:

```
gifts[3]
```

causing:

```
IndexOutOfRangeException
```

Correct:

```
i < gifts.Length
```

Counting Down with for

```
for (int retry = 3; retry > 0; retry--)
{
    Console.WriteLine(
        $"Retries remaining: {retry}");
}
```

Flow:

```
3
    →
2
    →
1
    →
0 condition fails
```

Different Increment Sizes

```
for (int i = 0; i < 10; i += 2)
{
    Console.WriteLine(i);
}
```

Output:

```
0
2
4
6
8
```

The iteration does not have to be:

```
i++
```

Multiple Variables in for

C# allows more complex forms:

```
for (int left = 0, right = 9;
     left < right;
     left++, right--)
{
}
```

Useful in algorithms such as:

```
two-pointer processing
palindrome checks
array comparisons
```

But use this only when it genuinely improves the algorithm.

foreach Loop

Definition

A `foreach` loop iterates through the elements of a sequence or collection.

Example:

```
decimal[] gifts =
{
    100m,
    250m,
    500m
};

foreach (decimal gift in gifts)
{
    Console.WriteLine(gift);
}
```

Flow:

```
First element
    →
process

Second element
    →
process

Third element
    →
process

No elements left
    →
stop
```

foreach Syntax

```
foreach (Type variable in collection)
{
}
```

Example:

```
foreach (decimal giftAmount in gifts)
{
}
```

The iteration variable represents the current element.

Fundraising Example

```
var giftAmounts =
    new List<decimal>
    {
        100m,
        200m,
        300m
    };

decimal total = 0m;

foreach (decimal amount in giftAmounts)
{
    total += amount;
}
```

Result:

```
600
```

foreach vs for

Use `foreach` when:

```
You simply need every element
```

Use `for` when:

```
You need the index

You need special counter behavior

You need to move backward

You need to skip using index arithmetic

You need specific position-based access
```

Example:

```
foreach (decimal gift in gifts)
{
    Console.WriteLine(gift);
}
```

is simpler than:

```
for (int i = 0; i < gifts.Length; i++)
{
    Console.WriteLine(gifts[i]);
}
```

when the index is not needed.

---

foreach and Index

`foreach` does not directly give you a numeric index.

If the index is important, a normal `for` loop is often clearer.

Example:

```
for (int i = 0; i < gifts.Count; i++)
{
    Console.WriteLine(
        $"Gift {i}: {gifts[i]}");
}
```

Modifying Collection During foreach

This is a critical rule.

Example:

```
var giftIds =
    new List<long>
    {
        1,
        2,
        3
    };

foreach (long id in giftIds)
{
    giftIds.Remove(id);
}
```

This is unsafe and normally results in an exception because the collection is being structurally modified during enumeration.

Think:

```
Enumerator expects collection structure
        →
collection changes
        →
enumeration becomes invalid
```

Safe Alternatives

If you need to remove values, you can use approaches such as:

```
giftIds.RemoveAll(id => id < 0);
```

or iterate carefully by index when appropriate.

Example:

```
for (int i = giftIds.Count - 1;
     i >= 0;
     i--)
{
    if (giftIds[i] < 0)
    {
        giftIds.RemoveAt(i);
    }
}
```

Why backwards?

If removing:

```
higher index first
```

you avoid shifting unprocessed earlier positions.

---

while Loop

Definition

A `while` loop repeats as long as its condition remains true.

Syntax:

```
while (condition)
{
}
```

Example:

```
int retryCount = 0;

while (retryCount < 3)
{
    Console.WriteLine(
        $"Attempt {retryCount + 1}");

    retryCount++;
}
```

while Flow

```
Check condition
    →
true?
    →
execute body
    →
check condition again

false?
    →
exit loop
```

Important:

```
Condition is checked before the first iteration.
```

Therefore, a `while` loop can execute:

```
zero times
```

Example Where while Executes Zero Times

```
int retryCount = 5;

while (retryCount < 3)
{
    Console.WriteLine("Retry");
}
```

Condition:

```
5 < 3
```

is false immediately.

The body never executes.

When to Use while

Use `while` when the number of repetitions is not known in advance.

Examples:

```
Retry until success or limit

Read until no more data

Process queue until empty

Wait while condition is true

Continue until user exits
```

Fundraising Retry Example

```
int retryCount = 0;
bool paymentSucceeded = false;

while (!paymentSucceeded
       &&
       retryCount < 3)
{
    retryCount++;

    Console.WriteLine(
        $"Payment attempt {retryCount}");

    paymentSucceeded =
        TryProcessPayment();
}
```

Flow:

```
Payment succeeded?
    →
No
    →
Retries remaining?
    →
Yes
    →
Try Payment
    →
Check again
```

do-while Loop

Definition

A `do-while` loop executes its body first, then checks the condition.

Syntax:

```
do
{
}
while (condition);
```

Example:

```
int attempt = 0;

do
{
    attempt++;

    Console.WriteLine(
        $"Attempt {attempt}");
}
while (attempt < 3);
```

do-while Flow

```
Execute Body
    →
Check Condition
    →
true?
    →
repeat

false?
    →
stop
```

Important difference:

```
do-while executes at least once
```

because the condition is checked after the first iteration.

---

while vs do-while

while

```
Check first
    →
then execute
```

Can execute:

```
0 times
```

do-while

```
Execute first
    →
then check
```

Executes:

```
at least 1 time
```

Example

```
int value = 10;

while (value < 5)
{
    Console.WriteLine("while");
}
```

prints nothing.

But:

```
do
{
    Console.WriteLine("do-while");
}
while (value < 5);
```

prints once.

When to Use do-while

Useful when the operation must happen before the continuation condition can be evaluated.

Examples:

```
Prompt user once

Perform initial attempt

Read one item then decide whether to continue
```

In modern backend applications, `do-while` is less common than `foreach`, `for`, or `while`, but it is still important to understand.

Nested Loops

A loop can exist inside another loop.

Example:

```
for (int campaign = 1;
     campaign <= 3;
     campaign++)
{
    for (int gift = 1;
         gift <= 2;
         gift++)
    {
        Console.WriteLine(
            $"Campaign {campaign}, Gift {gift}");
    }
}
```

Flow:

```
Campaign 1
    →
Gift 1
Gift 2

Campaign 2
    →
Gift 1
Gift 2

Campaign 3
    →
Gift 1
Gift 2
```

Nested Loop Cost

Suppose outer loop runs:

```
1000 times
```

and inner loop runs:

```
1000 times
```

Total body executions:

```
1,000 × 1,000
=
1,000,000
```

This becomes important when learning algorithm complexity later.

Nested loops are not automatically bad.

But understand their cost.

---

break Statement

Definition

`break` immediately exits the nearest loop or `switch`.

Example:

```
foreach (decimal amount in giftAmounts)
{
    if (amount >= 10_000m)
    {
        Console.WriteLine(
            "Principal gift found");

        break;
    }
}
```

Flow:

```
Read Gift
    →
Amount >= 10000?
    →
No → Continue

Yes
    →
break
    →
Exit Loop
```

When break Is Useful

Use `break` when:

```
Required item found

Further processing is unnecessary

Termination condition reached
```

Example:

```
bool found = false;

foreach (long giftId in giftIds)
{
    if (giftId == targetGiftId)
    {
        found = true;
        break;
    }
}
```

Later, LINQ can simplify many searches:

```
giftIds.Contains(targetGiftId);
```

But loops remain important foundational knowledge.

---

break Only Exits Nearest Loop

Example:

```
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        if (j == 1)
        {
            break;
        }
    }
}
```

`break` exits:

```
inner loop only
```

The outer loop continues.

---

continue Statement

Definition

`continue` skips the rest of the current iteration and moves to the next iteration.

Example:

```
foreach (decimal amount in giftAmounts)
{
    if (amount <= 0m)
    {
        continue;
    }

    Console.WriteLine(
        $"Processing {amount}");
}
```

Flow:

```
Current amount
    →
<= 0?
    →
Yes
    →
skip remaining body
    →
next element

No
    →
process gift
```

---

break vs continue

`break`:

```
Exit entire loop
```

`continue`:

```
Skip current iteration
Continue with next iteration
```

Example:

```
foreach (Gift gift in gifts)
{
    if (!gift.IsValid)
    {
        continue;
    }

    if (gift.RequiresManualReview)
    {
        break;
    }

    ProcessGift(gift);
}
```

---

return Statement

Definition

`return` exits the current method.

Example:

```
static void ProcessGift(
    decimal amount)
{
    if (amount <= 0m)
    {
        return;
    }

    Console.WriteLine(
        "Processing gift");
}
```

If amount is invalid:

```
return
    →
entire method ends
```

This differs from:

```
break
```

which exits only the loop or switch.

---

return with Value

Methods that produce values use:

```
return value;
```

Example:

```
static bool IsMajorGift(
    decimal amount)
{
    return amount >= 5000m;
}
```

We will study this deeply in the Methods topic.

---

break vs continue vs return

Keyword	Effect
`break`	Exit nearest loop or switch
`continue`	Skip current loop iteration
`return`	Exit current method

Infinite Loop

An **infinite loop** never naturally reaches a stopping condition.

Example:

```
while (true)
{
    Console.WriteLine("Running");
}
```

This continues indefinitely unless:

```
break
return
exception
process termination
cancellation
```

stops it.

Infinite Loop Can Be Intentional

Some background workers conceptually run:

```
while (!cancellationToken.IsCancellationRequested)
{
    await ProcessNextMessageAsync();
}
```

This is a controlled long-running loop.

Later, ASP.NET Core worker services and background services use patterns like this.

Accidental Infinite Loop

Bad:

```
int retryCount = 0;

while (retryCount < 3)
{
    Console.WriteLine("Retry");
}
```

Problem:

```
retryCount never changes
```

Condition remains:

```
0 < 3
```

forever.

Correct:

```
retryCount++;
```

inside the loop.

Loop Variable Scope

Example:

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

The variable:

```
i
```

exists only within the relevant `for` scope.

This would not work afterward:

```
// Console.WriteLine(i);
```

because `i` is no longer in scope.

foreach Variable Scope

```
foreach (Gift gift in gifts)
{
    Console.WriteLine(gift);
}
```

The `gift` variable belongs to the loop scope.

It is not available after the loop.

Variable Shadowing

Avoid confusing variable reuse in nested scopes.

For example, C# prevents certain conflicting local declarations precisely because confusing shadowing makes code difficult to reason about.

Prefer clear names:

```
foreach (Campaign campaign in campaigns)
{
    foreach (Gift gift in campaign.Gifts)
    {
    }
}
```

rather than vague names such as:

```
x
y
item
data
```

when meaningful domain names are available.

Iterating Arrays

```
decimal[] giftAmounts =
{
    100m,
    200m,
    300m
};
```

Using `for`:

```
for (int i = 0;
     i < giftAmounts.Length;
     i++)
{
    Console.WriteLine(
        giftAmounts[i]);
}
```

Using `foreach`:

```
foreach (decimal amount
         in giftAmounts)
{
    Console.WriteLine(amount);
}
```

Both are correct.

Iterating List

```
var giftIds =
    new List<long>
    {
        1001,
        1002,
        1003
    };

foreach (long giftId
         in giftIds)
{
    Console.WriteLine(giftId);
}
```

For collections, `foreach` is often the clearest choice.

Iterating Dictionary<TKey,TValue>

Preview example:

```
var totalsByCampaign =
    new Dictionary<long, decimal>
    {
        [1] = 1000m,
        [2] = 2500m
    };

foreach (var pair
         in totalsByCampaign)
{
    Console.WriteLine(
        $"{pair.Key}: {pair.Value}");
}
```

We will study dictionaries properly in the Collections topic.

Choosing for vs foreach vs while vs do-while

Use for when

```
You know/control the iteration count

You need an index

You need to iterate backward

You need custom counter steps
```

Example:

```
for (int i = 0; i < gifts.Count; i++)
{
}
```

Use foreach when

```
You want every element in a sequence

You do not need index manipulation
```

Example:

```
foreach (Gift gift in gifts)
{
}
```

This is extremely common in normal C# application code.

Use while when

```
Iteration count is unknown

Continue until a condition changes
```

Example:

```
while (!paymentSucceeded
       && retryCount < 3)
{
}
```

Use do-while when

```
Body must execute at least once
```

Example:

```
do
{
    TryOperation();
}
while (shouldRetry);
```

Practical Fundraising Example — Process Valid Gifts

```
decimal[] giftAmounts =
{
    500m,
    -100m,
    1000m,
    0m,
    2500m
};

foreach (decimal amount
         in giftAmounts)
{
    if (amount <= 0m)
    {
        continue;
    }

    Console.WriteLine(
        $"Processing gift: {amount:C}");
}
```

Flow:

```
500
    →
valid
    →
process

-100
    →
invalid
    →
continue

1000
    →
valid
    →
process

0
    →
invalid
    →
continue

2500
    →
valid
    →
process
```

Practical Example — Find First Major Gift

```
decimal[] giftAmounts =
{
    100m,
    500m,
    1500m,
    7500m,
    12000m
};

decimal? firstMajorGift = null;

foreach (decimal amount
         in giftAmounts)
{
    if (amount >= 5000m)
    {
        firstMajorGift = amount;
        break;
    }
}
```

Result:

```
7500
```

Because the loop stops at the first qualifying gift.

Practical Example — Calculate Total Giving

```
decimal[] gifts =
{
    100m,
    250m,
    400m
};

decimal totalGiving = 0m;

foreach (decimal gift
         in gifts)
{
    totalGiving += gift;
}
```

Flow:

```
Start total = 0
    →
+100
    →
100
    →
+250
    →
350
    →
+400
    →
750
```

Result:

```
750
```

Practical Example — Count Major Gifts

```
decimal[] gifts =
{
    500m,
    5000m,
    7500m,
    200m,
    10000m
};

int majorGiftCount = 0;

foreach (decimal gift
         in gifts)
{
    if (gift >= 5000m)
    {
        majorGiftCount++;
    }
}
```

Result:

```
3
```

Practical Example — Gift Classification

```
string category =
    giftAmount switch
    {
        >= 10_000m
            => "Principal Gift",

        >= 5_000m
            => "Major Gift",

        >= 1_000m
            => "Leadership Gift",

        > 0m
            => "Standard Gift",


        _
            => "Invalid Gift"
    };
```

This is much cleaner than a long assignment-based `if` chain when the purpose is simply:

```
one input
    →
one classification
```

Practical Example — Batch Processing

Suppose we have:

```
var giftIds =
    Enumerable.Range(1, 100)
        .Select(x => (long)x)
        .ToList();
```

We want 25 gifts per batch.

Simple index-based flow:

```
const int batchSize = 25;

for (int i = 0;
     i < giftIds.Count;
     i += batchSize)
{
    int remaining =
        giftIds.Count - i;

    int currentBatchSize =
        Math.Min(
            batchSize,
            remaining);

    Console.WriteLine(
        $"Starting batch at {i}, "
        + $"size {currentBatchSize}");
}
```

Conceptually:

```
0   → 25 rows
25  → 25 rows
50  → 25 rows
75  → 25 rows
```

This becomes important later for:

```
database imports
API integrations
message processing
```

Practical Example — Retry with break

```
const int maxRetries = 3;

bool succeeded = false;

for (int attempt = 1;
     attempt <= maxRetries;
     attempt++)
{
    Console.WriteLine(
        $"Attempt {attempt}");

    succeeded =
        TryProcessPayment();

    if (succeeded)
    {
        break;
    }
}
```

Flow:

```
Attempt 1
    →
success?
    →
No
    →
Attempt 2
    →
success?
    →
Yes
    →
break
```

Avoid Blocking Retry Loops in Real Web APIs

A beginner might write:

```
while (!success)
{
    success =
        CallExternalPaymentApi();
}
```

This can be dangerous because:

```
No retry limit

Potential infinite loop

External service may remain down

Request thread/resources may stay occupied

No delay/backoff
```

Production retry strategies often require:

```
Maximum attempts

Timeouts

Cancellation

Delay

Exponential backoff

Jitter

Circuit breaker
```

We will study these later under external HTTP integrations and resilience.

Conditions and Null Handling

Example:

```
if (campaign is null)
{
    return;
}

if (!campaign.IsActive)
{
    return;
}

ProcessCampaign(campaign);
```

This style is usually clearer than:

```
if (campaign is not null)
{
    if (campaign.IsActive)
    {
        ProcessCampaign(campaign);
    }
}
```

Guard clauses reduce nesting.

Pattern Matching in if

Modern C# allows patterns in conditions.

Example:

```
if (giftAmount is > 0m and < 1000m)
{
    Console.WriteLine(
        "Standard gift");
}
```

Another:

```
if (gift is not null)
{
}
```

Another:

```
if (value is string text)
{
    Console.WriteLine(text);
}
```

Pattern matching will receive a dedicated advanced topic later.

For now understand that `if` conditions are not limited to simple comparison operators.

Multiple Conditions with Pattern Matching

Example:

```
if (giftAmount is >= 1000m
    and < 5000m)
{
    Console.WriteLine(
        "Leadership Gift");
}
```

This can sometimes read naturally.

Compare:

```
if (giftAmount >= 1000m
    &&
    giftAmount < 5000m)
{
}
```

Both are valid.

Choose the clearer representation for the codebase.

Common Mistake — Wrong Condition Order

Bad:

```
if (giftAmount >= 1000m)
{
    category = "Leadership";
}
else if (giftAmount >= 5000m)
{
    category = "Major";
}
```

The second branch is unreachable for values above 5000 because they already match:

```
>= 1000
```

Correct:

```
highest threshold
    →
lower thresholds
```

Common Mistake — Missing Braces

Bad:

```
if (paymentSucceeded)
    MarkPaymentComplete();
    SendReceipt();
```

`SendReceipt()` always runs.

Better:

```
if (paymentSucceeded)
{
    MarkPaymentComplete();
    SendReceipt();
}
```

Common Mistake — Assignment vs Equality

Wrong idea:

```
// if (status = 1)
```

Correct comparison:

```
if (status == 1)
{
}
```

Common Mistake — Infinite while Loop

```
int count = 0;

while (count < 10)
{
    Console.WriteLine(count);
}
```

`count` never changes.

Correct:

```
count++;
```

Common Mistake — Off-by-One Error

Wrong:

```
for (int i = 0;
     i <= gifts.Count;
     i++)
```

Correct:

```
for (int i = 0;
     i < gifts.Count;
     i++)
```

when indexing from zero.

Common Mistake — Modifying Collection in foreach

Avoid:

```
foreach (Gift gift in gifts)
{
    if (!gift.IsValid)
    {
        gifts.Remove(gift);
    }
}
```

The collection's enumerator can become invalid.

Use a suitable collection operation or a controlled index-based approach.

Common Mistake — Too Much Nesting

Hard to read:

```
if (user != null)
{
    if (user.IsActive)
    {
        if (gift != null)
        {
            if (gift.TotalAmount > 0m)
            {
                ProcessGift();
            }
        }
    }
}
```

Better:

```
if (user is null)
{
    return;
}

if (!user.IsActive)
{
    return;
}

if (gift is null)
{
    return;
}

if (gift.TotalAmount <= 0m)
{
    return;
}

ProcessGift();
```

Common Mistake — continue Everywhere

`continue` can simplify loops, but excessive use can make control flow hard to follow.

Good:

```
foreach (Gift gift in gifts)
{
    if (!gift.IsValid)
    {
        continue;
    }

    ProcessGift(gift);
}
```

Harder:

```
multiple continue points
nested conditions
multiple state mutations
```

Keep loop flow understandable.

Common Mistake — break When return Is Needed

Suppose:

```
foreach (Gift gift in gifts)
{
    if (gift.Id == targetId)
    {
        ProcessGift(gift);
        break;
    }
}

DoMoreMethodWork();
```

`break` exits only the loop.

If the entire method should end, use:

```
return;
```

when appropriate.

Performance Considerations

For ordinary application logic, choosing between:

```
for
foreach
while
```

should primarily be about:

```
clarity
correctness
algorithm
```

not tiny performance differences.

In backend applications, larger performance costs usually come from:

```
Database calls

Network requests

Serialization

Large allocations

Poor algorithms

Repeated collection enumeration
```

rather than the basic loop syntax itself.

Algorithmic Cost Matters

This:

```
foreach (Gift gift in gifts)
{
}
```

runs roughly once per gift.

If there are:

```
1,000,000 gifts
```

there are:

```
1,000,000 iterations
```

Now:

```
foreach (Gift gift in gifts)
{
    foreach (Campaign campaign
             in campaigns)
    {
    }
}
```

If both contain:

```
1,000,000 elements
```

the theoretical combinations become enormous.

The loop syntax is not the problem.

The algorithm is.

Later, collection and performance topics will study:

```
O(n)

O(n²)

dictionary lookups

hash sets
```

more deeply.

Avoid Database Calls Inside Large Loops

Very important for future ASP.NET/EF Core development.

Bad concept:

```
foreach (long giftId in giftIds)
{
    Gift gift =
        await dbContext.Gifts
            .FindAsync(giftId);
}
```

If:

```
1000 Gift IDs
```

this could potentially cause:

```
1000 database operations
```

depending on state/query behavior.

Better approaches might include:

```
one query for all IDs

batching

set-based database operations
```

Later EF Core and SQL topics will cover this in depth.

Async Loop Preview

Later you may write:

```
foreach (Gift gift in gifts)
{
    await ProcessGiftAsync(gift);
}
```

This processes sequentially.

Do not immediately replace it with:

```
Task.WhenAll(...)
```

without considering:

```
database concurrency

API rate limits

memory

thread safety

connection limits
```

Async and concurrency will be studied in dedicated roadmap topics.

Security Considerations

Conditional logic often controls security.

Example:

```
if (isAdministrator
    ||
    isOwner)
{
    AllowEdit();
}
```

A small logic mistake can become an authorization vulnerability.

Suppose requirement is:

```
User must be active
AND
must be either Admin or Owner
```

Correct:

```
if (isActive
    &&
    (isAdministrator || isOwner))
{
    AllowEdit();
}
```

Not:

```
if (isActive
    &&
    isAdministrator
    ||
    isOwner)
{
    AllowEdit();
}
```

because precedence groups the second expression differently.

Security conditions should be explicit and preferably centralized in authorization policies later.

Conditions Are Not a Substitute for Authorization

Do not spread security logic everywhere:

```
if (user.Role == "Admin")
```

through dozens of controllers.

Later ASP.NET Core will provide:

```
Authentication

Claims

Policies

Authorization handlers
```

But understanding Boolean conditions is required before those concepts make sense.

Practical Coding Exercise — Even or Odd

Requirement:

> Determine whether a number is even.

```
int number = 10;

if (number % 2 == 0)
{
    Console.WriteLine("Even");
}
else
{
    Console.WriteLine("Odd");
}
```

Practical Coding Exercise — Largest of Two Numbers

```
int first = 10;
int second = 25;

if (first > second)
{
    Console.WriteLine(first);
}
else
{
    Console.WriteLine(second);
}
```

Practical Coding Exercise — Largest of Three Numbers

```
int a = 10;
int b = 30;
int c = 20;

int largest;

if (a >= b && a >= c)
{
    largest = a;
}
else if (b >= a && b >= c)
{
    largest = b;
}
else
{
    largest = c;
}
```

Result:

```
30
```

Practical Coding Exercise — Sum Array

```
int[] values =
{
    10,
    20,
    30,
    40
};

int total = 0;

foreach (int value in values)
{
    total += value;
}
```

Result:

```
100
```

Practical Coding Exercise — Find Maximum

```
int[] values =
{
    10,
    50,
    20,
    90,
    40
};

int maximum =
    values[0];

for (int i = 1;
     i < values.Length;
     i++)
{
    if (values[i] > maximum)
    {
        maximum = values[i];
    }
}
```

Result:

```
90
```

Flow:

```
Start max = 10
    →
50 > 10 → max = 50
    →
20 > 50? No
    →
90 > 50 → max = 90
    →
40 > 90? No
    →
Final max = 90
```

Practical Coding Exercise — Count Positive Values

```
int[] values =
{
    -10,
    5,
    0,
    20,
    -4,
    30
};

int positiveCount = 0;

foreach (int value in values)
{
    if (value > 0)
    {
        positiveCount++;
    }
}
```

Result:

```
3
```

Practical Coding Exercise — Reverse Output

```
string text = "GIFT";

for (int i = text.Length - 1;
     i >= 0;
     i--)
{
    Console.Write(text[i]);
}
```

Output:

```
TFIG
```

This starts preparing you for the interview question:

> Reverse a string without built-in reverse methods.

Practical Coding Exercise — Palindrome Preview

```
string text = "LEVEL";

bool isPalindrome = true;

for (int left = 0,
         right = text.Length - 1;
     left < right;
     left++, right--)
{
    if (text[left] != text[right])
    {
        isPalindrome = false;
        break;
    }
}
```

Flow:

```
L == L
    →
E == E
    →
middle reached
    →
Palindrome
```

This combines:

```
for
comparison
multiple loop variables
break
```

Interview Questions

What is control flow?

Control flow is the order in which program statements execute.

Conditions and loops allow the program to:

```
choose different paths
and
repeat work
```

What is an if statement?

An `if` statement executes a block only when its Boolean condition evaluates to `true`.

Example:

```
if (giftAmount > 0m)
{
    ProcessGift();
}
```

What is the difference between if and if-else?

`if` executes code only for the true condition.

`if-else` provides one branch for true and another for false.

What is an else-if chain?

An `else if` chain tests several conditions in sequence.

The first matching branch executes and the remaining branches are skipped.

Why does condition order matter in else-if?

Because evaluation happens from top to bottom.

A broad condition placed too early can make more specific later conditions unreachable.

What is a guard clause?

A guard clause checks invalid or exceptional conditions early and exits the current flow.

Example:

```
if (giftAmount <= 0m)
{
    return;
}
```

Guard clauses can reduce deep nesting.

What is a switch statement?

A `switch` selects one branch based on a value or pattern.

It is useful when one input has several distinct cases.

What is a switch expression?

A switch expression maps an input to a resulting expression.

Example:

```
string status =
    statusCode switch
    {
        1 => "Pending",
        2 => "Complete",
        _ => "Unknown"
    };
```

When should you use switch instead of if?

Use `switch` when one value is being matched against multiple alternatives.

Use `if` when logic involves ranges, multiple variables, or more complex Boolean conditions.

Modern pattern matching makes the boundary more flexible, so readability is the main guide.

What is a for loop?

A `for` loop repeats code using initialization, continuation condition, and iteration expressions.

It is commonly used when a loop counter or index is needed.

What is foreach?

`foreach` iterates over each element of an enumerable sequence.

It is usually the clearest loop when you simply need to process every element.

Difference between for and foreach?

`for` gives direct control over the index and iteration step.

`foreach` directly provides each element and is generally simpler when the index is not needed.

What is a while loop?

A `while` loop repeatedly executes while its condition remains true.

The condition is checked before each iteration.

It can execute zero times.

What is a do-while loop?

A `do-while` executes its body first and evaluates the condition afterward.

Therefore it executes at least once.

What is the difference between while and do-while?

`while`:

```
check first
```

`do-while`:

```
execute first
```

Therefore only `do-while` guarantees one execution.

What does break do?

`break` exits the nearest loop or switch immediately.

What does continue do?

`continue` skips the remainder of the current loop iteration and proceeds to the next iteration.

What does return do?

`return` exits the current method.

If the method returns a value, `return` can also provide that result.

Difference between break and return?

`break` exits only the nearest loop/switch.

`return` exits the whole current method.

What is an infinite loop?

An infinite loop is a loop whose termination condition never becomes false.

Example:

```
while (true)
{
}
```

It may be intentional in controlled background-processing scenarios or accidental due to incorrect loop logic.

What is an off-by-one error?

An off-by-one error occurs when loop boundaries include one iteration too many or too few.

A classic example:

```
i <= array.Length
```

when valid indexes end at:

```
Length - 1
```

Can you modify a List while iterating with foreach?

Structural modification of a normal `List<T>` during its enumeration is generally invalid and can cause an `InvalidOperationException`.

Use an appropriate alternative such as filtering, `RemoveAll`, or controlled index-based modification.

What is a nested loop?

A nested loop is a loop inside another loop.

The inner loop runs for each relevant iteration of the outer loop.

This can significantly increase total work.

Which loop is most common in business C# code?

`foreach` is extremely common for processing collections.

`for` is useful for index-based work.

`while` is common when repetition depends on a changing condition rather than a known count.

There is no universally best loop.

Best Practices

Use braces with conditional and loop bodies even for single statements when possible.

Prefer:

```
simple conditions
```

over deeply nested logic.

Use guard clauses to reduce unnecessary nesting where they clarify the flow.

Check more specific conditions before broader ones.

Prefer `foreach` when you only need each element.

Use `for` when index/control is required.

Use `while` when the number of repetitions is unknown.

Use `do-while` only when at least one execution is genuinely required.

Avoid accidental infinite loops.

Use `break` when further iterations are unnecessary.

Use `continue` to skip invalid items when that makes the loop clearer.

Do not structurally modify normal collections during a `foreach`.

Avoid large database/network operations inside loops unless the architecture requires them.

Be cautious with nested loops over large collections.

Keep authorization conditions explicit and easy to review.

Do not use giant `if`/`else` or `switch` structures when polymorphism, strategy, mappings, or dedicated business objects would better model the problem.

Quick Revision

Concept	Meaning
`if`	Execute when condition is true
`else`	Execute when preceding `if` is false
`else if`	Check another condition
Guard clause	Exit early for invalid state
`switch`	Choose a branch based on value/pattern
switch expression	Map input to result expression
`for`	Counter/index-controlled loop
`foreach`	Iterate each sequence element
`while`	Check condition before each iteration
`do-while`	Execute once before checking
`break`	Exit nearest loop/switch
`continue`	Skip current iteration
`return`	Exit current method
Nested loop	Loop inside another loop
Infinite loop	Loop that does not naturally terminate
Off-by-one	Loop boundary one position wrong

Complete Fundraising Example

```
decimal[] giftAmounts =
{
    100m,
    7500m,
    -50m,
    12000m,
    2500m
};

decimal totalValidGiving = 0m;
int majorGiftCount = 0;

foreach (decimal amount
         in giftAmounts)
{
    if (amount <= 0m)
    {
        continue;
    }

    totalValidGiving += amount;

    string category =
        amount switch
        {
            >= 10_000m
                => "Principal Gift",

            >= 5_000m
                => "Major Gift",

            >= 1_000m
                => "Leadership Gift",

            _
                => "Standard Gift"
        };

    Console.WriteLine(
        $"{amount:C} - {category}");

    if (amount >= 5000m)
    {
        majorGiftCount++;
    }
}

Console.WriteLine(
    $"Total giving: {totalValidGiving:C}");

Console.WriteLine(
    $"Major/principal gifts: {majorGiftCount}");
```

Flow:

```
Read Gift
    →
Amount <= 0?
    →
Yes
    →
Skip

No
    →
Add to Total
    →
Classify Gift
    →
Major or Principal?
    →
Yes
    →
Increment MajorGiftCount
    →
Next Gift
```

For the example data:

```
100
    →
Standard

7500
    →
Major

-50
    →
Skipped

12000
    →
Principal

2500
    →
Leadership
```

The program demonstrates how:

```
conditions
switch expressions
foreach
continue
incrementing
business classification
```

work together in real application logic.

Complete Topic Flow

```
Input / Current State
    →
Evaluate Boolean Condition
    →
Choose Path

if / else
or
switch
    →
Need Repetition?
    →
No
    →
Continue Program

Yes
    →
Choose Loop

Known counter/index?
    →
for

Every collection element?
    →
foreach

Unknown count, condition first?
    →
while

Must run at least once?
    →
do-while
    →
During Loop

Need skip current item?
    →
continue

Need stop loop?
    →
break

Need stop whole method?
    →
return
    →
Program continues
```

Fundraising processing:

```
Gift Collection
    →
foreach Gift
    →
Valid?
    →
No → continue

Yes
    →
Classify Gift
    →
Add to Total
    →
Need special handling?
    →
Process
    →
Next Gift
```

Conditional statements and loops are the foundation for nearly every business workflow you will write later in C#, including validation, collection processing, retry logic, authorization decisions, data transformations, API behavior, and algorithmic coding problems.

Next roadmap topic: Methods and Parameter Passing.